PRELIMINARY EXAMINATIONS 2026

Course 3: Basics of Web Development

A Complete Educational Manual on Client-Server Architecture, Responsive Layout Engineering, and Interactive Client-Side Logic using HTML5, CSS3, and JavaScript

COURSE CODE
CS-103

ACADEMIC YEAR
2026

PUBLISHER
Preliminary Examinations Press

TARGET AUDIENCE
Undergraduate Computer Science & Information Systems Majors

Table of Contents

MODULE 1

Web & HTML Fundamentals

1.1 Client-Server Architecture & The Web

The World Wide Web operates on a distributed client-server model. A **Client** (typically a web browser like Chrome or Firefox) initiates requests to the network, and a **Server** (a remote host storing website resources) processes those requests, returning the required documents.

The communication relies on the **HTTP (Hypertext Transfer Protocol)** or its secure version **HTTPS**. The process involves a two-way transaction: an HTTP Request (sent by client containing parameters, headers, and request methods like GET/POST) and an HTTP Response (sent by the server containing response headers, status codes like 200 OK or 404 Not Found, and the HTML document body).

Figure 1.1: The client-server communication lifecycle over HTTP protocol.

1.2 Document Structure & Text Layouts

HTML (HyperText Markup Language) defines the semantic structure of a web page. A document begins with the declaration `<!DOCTYPE html>`, which informs the browser to render the page in standard HTML5 mode. The root element is `<html>`, containing a `<head>` (metadata, title, styles) and `<body>` (renderable content visible to the user).

Headings and Paragraphs

HTML provides six levels of document headings, from `<h1>` (most important) to `<h6>` (least important), alongside paragraphs `<p>`:

```
<h1>Main Chapter Title</h1>
<h2>Subheading 1.2</h2>
<p>This is a paragraph containing a <strong>bolded term</strong> and <em>italicize
```

VISUAL OUTPUT

Main Chapter Title

Subheading 1.2

This is a paragraph containing a **bolded term** and *italicized text*.

1.3 Hyperlinks, Lists, Tables & Multimedia

HTML lists can be unordered (bulleted, `<ul>`) or ordered (numbered, `<ol>`). Data grids are built using tables, and navigation links use the anchor tag `<a>`:

```
<h3>Key Concepts & Navigation</h3>
<ul>
    <li>Visit <a href="https://example.com" target="_blank">Example Website</a></l
    <li>Contiguous memory storage</li>
</ul>

<table style="width:100%; border: 1px solid #15803d; border-collapse: collapse;">
    <tr style="background-color: #f0fdf4;">
        <th style="border: 1px solid #15803d; padding: 4px;">Element</th>
        <th style="border: 1px solid #15803d; padding: 4px;">Function</th>
    </tr>
    <tr>
        <td style="border: 1px solid #15803d; padding: 4px;">Anchor (&lt;a&gt;)</t
        <td style="border: 1px solid #15803d; padding: 4px;">Hyperlink routing</td
    </tr>
</table>
```

VISUAL OUTPUT

Key Concepts & Navigation

- Visit Example Website
- Contiguous memory storage

Element	Function
Anchor (<a>)	Hyperlink routing

1.4 Semantic HTML Structure

Semantic elements clearly describe their meaning to both the browser and the developer. Using semantic tags like `<header>`, `<nav>`, `<article>`, `<section>`, and `<footer>` instead of nesting generic `<div>` blocks improves search engine optimization (SEO), code readability, and accessibility for screen readers.

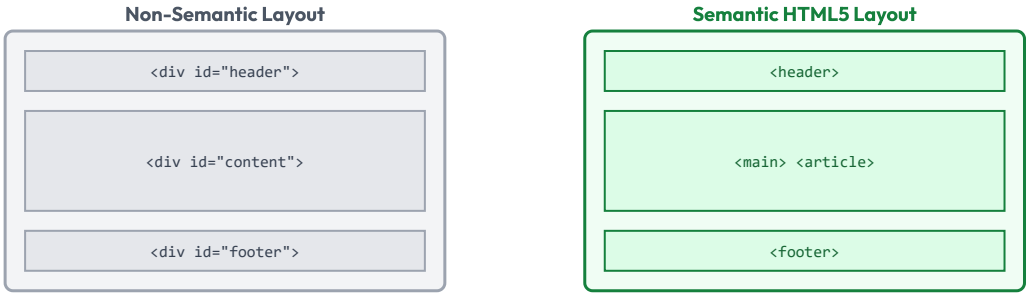

Figure 1.2: Structuring pages using divs vs. standardized HTML5 semantic tags.

Module 1 - Exercises & Review Tasks

1 Debugging Task: Broken Tags and Elements

Find and correct all syntactic errors in the following HTML document segment:

```
<section>
    <h1>Popular Coding Languages<h1>
    <p>HTML and CSS are used to structure websites.
    <img src="code.jpg" alt="syntax snippet"
    <a href="details.html">View More Info</p></a>
</section>
```

2 Guess the Output: Complex Grid Tables

Trace the row and cell alignments and sketch the visual table produced by this code:

```
<table border="1">
    <tr>
        <th colspan="2">Sales Summary</th>
    </tr>
    <tr>
        <td>Q1</td>
        <td>$12,000</td>
    </tr>
</table>
```

3 Theory Review Questions

Answer the following questions in detail:

- Describe the client-server architecture model. What happens step-by-step when a user types a URL into a browser address bar?
- Explain the differences between standard block-level elements (e.g., `<div>`, `<p>`) and inline elements (e.g., `<span>`, `<a>`) in terms of flow and layout.
- What are HTTP status codes? List four common status codes and explain their meanings.
- Why are semantic HTML tags preferred over generic div tags for web layout structures? Discuss accessibility and SEO implications.

Module 1 - Practical Lab Work

Lab 1.1: Personal Portfolio Homepage Skeleton

Create a single, standards-compliant HTML page containing your professional bio. Structure the document using semantic HTML5 elements (`<header>` , `<nav>` , `<main>` , `<section>` , and `<footer>`).

Requirements: Include a navigation list with jump-links to different sections of the page, a data table showcasing your skills and proficiency levels, and an inline profile image with appropriate alt text.

MODULE 2

Forms & Advanced HTML

2.1 Web Forms and Input Architecture

Web forms are the primary way websites collect data from users. A form is defined using the `<form>` tag, which encloses various input elements.

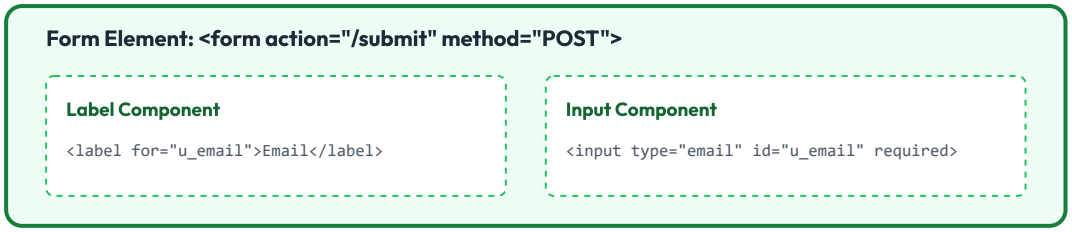

Figure 2.1: Semantic pairing of labels and input fields inside an HTML form.

2.2 Label Binding, Fieldsets & Validation

Label Binding: Labels should be bound to their inputs using the `for` attribute, which must match the input's `id` attribute. This allows users to click the label to focus the input field, improving accessibility for screen readers and touch screens.

Fieldset and Legend: Used to group related form controls together. The `<legend>` tag defines a caption for the group:

```
<form action="#" method="POST">
    <fieldset style="border: 1px solid #15803d; padding: 10px; border-radius: 4px;
        <legend style="color:#15803d; font-weight:bold; padding: 0 5px;">User Auth
        <label for="username" style="display:block; margin-bottom: 4px;">Username:
        <input type="text" id="username" name="user" required style="border: 1px s
    </fieldset>
</form>
```

VISUAL OUTPUT

User Authentication

Username:

2.3 Meta Tags, SEO Basics & Favicons

Meta tags are placed in the `<head>` of an HTML document to provide search engines and browsers with metadata about the page. This metadata is crucial for Search Engine Optimization (SEO).

- `<meta charset="UTF-8">`: Declares the character encoding for the document.
- `<meta name="viewport" content="width=device-width, initial-scale=1.0">`: Configures the page's viewport size to scale correctly on mobile devices.
- `<meta name="description" content="...">`: Provides search engines with a summary description of the page, often displayed in search results.
- `<link rel="icon" type="image/x-icon" href="favicon.ico">`: Links to the favicon, the icon displayed in the browser tab.

2.4 Accessibility (ARIA & Screen Readers)

Web accessibility (a11y) ensures that websites can be used by everyone, including people with visual, auditory, motor, or cognitive disabilities.

- **Alt Text:** All images must include an `alt` attribute describing the image content. Screen readers read this description aloud to visually impaired users.
- **WAI-ARIA:** Accessible Rich Internet Applications (ARIA) attributes provide extra context to screen readers when semantic HTML is not enough (e.g., using `aria-required="true"` or `role="button"`).

Module 2 - Exercises & Review Tasks

1

Debugging Task: Form Label and Validation Bugs

Find the issues causing broken validation and inaccessible labels in this HTML form:

```
<form action="/save">
    <label for="user">Full Name:</label>
    <input type="txt" name="name" required>

    <label for="mail">Email Address:</label>
    <input type="email" id="email">

    <button type="button">Submit Form</button>
</form>
```

2

Guess the Output: Native HTML validation warnings

What happens when a user attempts to submit a form containing the following input without entering any text, and when they enter "john-doe"? Explain the browser's response.

```
<input type="email" id="user-email" required>
```

3

Theory Review Questions

Answer the following questions in detail:

- Explain the structural purpose of the `<fieldset>` and `<legend>` elements in form layouts.
- What is the difference between the GET and POST request methods when submitting form data to a server?
- Define Search Engine Optimization (SEO). How do meta tags inside the `<head>` block affect search engine indexing?

- What is the purpose of the WAI-ARIA standards? How do attributes like `aria-label` improve a site's accessibility?

Module 2 - Practical Lab Work

Lab 2.1: Course Registration Form with validation

Create a complete C++ Course Registration HTML form. Organize inputs using separate `<fieldset>` categories: Personal Data and Course Options.

Requirements: Use email, telephone, checkbox selection lists, date picker inputs, and submit buttons. Enforce native validation using attributes like `required`, `pattern`, and character limits.

MODULE 3

CSS Fundamentals

3.1 CSS Rulesets, Selectors & Cascading Rules

CSS (Cascading Style Sheets) controls the presentation and visual layout of a web page. A CSS ruleset consists of a selector and a declaration block:

```
/* CSS Selector targets all paragraph elements */
p {
    color: #15803d;       /* Property: Value */
    font-size: 11pt;
    line-height: 1.6;
}
```

CSS Selector Priority & Specificity

When multiple CSS declarations target the same HTML element, the browser resolves conflicts using specificity and cascade rules. Specificity is calculated as a hierarchy:

1. **Inline Styles:** Styled directly on the element using the `style` attribute (highest priority).
2. **IDs:** Selectors targeting an element ID (e.g., `#header`).
3. **Classes, Attributes, and Pseudo-classes:** Selectors targeting classes (e.g., `.button`) or attributes.
4. **Elements (Tags):** Selectors targeting element tags (e.g., `div`, `p`) (lowest priority).

3.2 The CSS Box Model

Every HTML element is rendered as a rectangular box. The CSS Box Model defines the spacing and boundaries of this box, consisting of four nested layers:

1. **Content:** The actual text, images, or media inside the element.
2. **Padding:** Transparent spacing around the content, inside the border.
3. **Border:** A visible boundary wrapping the padding and content.
4. **Margin:** Transparent spacing outside the border, used to separate the element from neighboring elements.

Figure 3.1: The CSS Box Model hierarchy defining margin, border, padding, and content dimensions.

3.3 Typography, Colors, and Backgrounds

CSS is used to style text typography, colors, and backgrounds. Let's look at a code snippet styling a card component:

```
.card {
    background-color: #f0fdf4;
    border: 2px solid #15803d;
    border-radius: 8px;
    padding: 15px;
    color: #166534;
    font-family: 'Outfit', sans-serif;
    box-shadow: 0 4px 6px rgba(0,0,0,0.05);
}
```

VISUAL OUTPUT

Web Design Basics

Learn how to build responsive layouts using HTML and CSS grids.

3.4 Layout Position, Overflow & Z-index

Position Property: Controls how an element is positioned on the page:

- `static`: The default position. Elements render in their normal order in the document flow.
- `relative`: The element is positioned relative to its normal position, without affecting the layout of neighboring elements.
- `absolute`: The element is removed from the normal document flow and positioned relative to its closest positioned ancestor.
- `fixed`: The element is positioned relative to the browser viewport, remaining in place when the page is scrolled.

Z-index: Controls the stack order of overlapping elements (elements with a larger z-index are drawn in front of elements with a smaller z-index). It only works on positioned elements (relative, absolute, fixed).

Module 3 - Exercises & Review Tasks

1

Debugging Task: Selector Priority Conflicts

Determine why the text in the following paragraph displays in red instead of green, and fix the selector:

```
<style>
    .green-text { color: green; }
    #alert p { color: red; }
</style>
<div id="alert">
    <p class="green-text">This text should be green!</p>
</div>
```

2

Guess the Output: Box Model Calculations

Calculate the total rendered width and height of this element in pixels, assuming `box-sizing: content-box`:

```
.box {
    width: 250px;
    height: 100px;
    padding: 20px;
    border: 5px solid green;
    margin: 15px;
}
```

3

Theory Review Questions

Answer the following questions in detail:

- Explain the CSS Box Model. How does setting `box-sizing: border-box` change width and height calculations compared to `content-box`?
- Differentiate between the `relative`, `absolute`, and `fixed` values of the CSS `position` property.
- What is CSS Specificity? How does the browser calculate priority when resolving styling conflicts?

- What is the function of the `z-index` property? What are the prerequisites for an element to respect a z-index declaration?

Module 3 - Practical Lab Work

Lab 3.1: Profile Card Component

Design a card component containing an avatar image, a name heading, a bio description, and a button. Use CSS to style the layout.

Requirements: Apply a custom color palette, set padding and margins using the Box Model, round the card corners using `border-radius`, and add a subtle box-shadow hover effect.

MODULE 4

Responsive Web Design

4.1 CSS Flexbox Layouts

CSS Flexbox is a one-dimensional layout model designed for laying out items in rows or columns. It dynamically resizes items to fill the available space, making it easy to align elements and distribute space within a container.

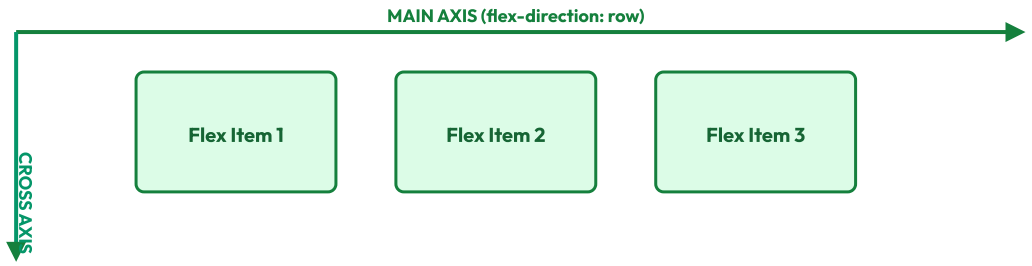

Figure 4.1: Flexbox coordinate axis showing main and cross alignments.

Let's look at a code snippet demonstrating basic Flexbox centering:

```
.flex-container {
    display: flex;
    justify-content: center; /* Centers items along the main axis */
    align-items: center;     /* Centers items along the cross axis */
    gap: 15px;
}
```

VISUAL OUTPUT

4.2 CSS Grid Systems

Unlike Flexbox, CSS Grid is a two-dimensional layout system designed for managing rows and columns simultaneously. This makes it ideal for building complex page structures and

alignments.

```
.grid-container {
    display: grid;
    grid-template-columns: repeat(3, 1fr); /* Creates three equal-width columns */
    gap: 10px;
}
```

VISUAL OUTPUT

4.3 Media Queries & Mobile-First Design

Responsive web design ensures that a website looks good on all devices, from mobile phones to desktop monitors. This is achieved using **Media Queries**, which apply CSS rules only when the browser viewport matches specific conditions:

```
/* Default styles target mobile devices (Mobile-First) */
body {
    background-color: #ffffff;
    font-size: 10pt;
}

/* Styles applied when the viewport width is 768px or larger */
@media (min-width: 768px) {
    body {
        background-color: #f0fdf4; /* Light green tint on tablet/desktop */
        font-size: 11pt;
    }
}
```

4.4 CSS Transitions & Keyframe Animations

Transitions: Smoothly animate changes to CSS properties over a specified duration (e.g., changing a button's background color on hover).

Animations: Create complex, multi-step animations using keyframes:

```
.btn-animate {
    background-color: #15803d;
    color: white;
    padding: 10px 20px;
    border: none;
    border-radius: 4px;
    transition: background-color 0.3s ease;
}
.btn-animate:hover {
    background-color: #166534;
}
```

VISUAL OUTPUT

Module 4 - Exercises & Review Tasks

1

Debugging Task: Grid Wrap & Column Collapses

Correct the CSS rule so that the grid displays three equal-width columns without overflowing the screen width:

```
.grid {
    display: grid;
    grid-template-columns: 33% 33% 33%;
    gap: 20px; /* Gap causes overflow when using percentages! */
}
```

2

Guess the Output: Flexbox Wrap Logic

Determine how elements align in a container with a width of 300px when the child item width is set to 120px. Does wrapping occur? Sketch the layout.

```
.container {
    display: flex;
    flex-wrap: wrap;
    gap: 10px;
}
```

3

Theory Review Questions

Answer the following questions in detail:

- Differentiate between CSS Flexbox (1D) and CSS Grid (2D) layout models. When should you use which?
- Explain the concept of "Mobile-First Design". What are the benefits of using `min-width` instead of `max-width` in media queries?
- What is the difference between a CSS Transition and a CSS Keyframe Animation?
- Describe the function of CSS variables (custom properties). How do they improve style maintenance across large web projects?

Module 4 - Practical Lab Work

Lab 4.1: Responsive Navbar & Gallery Grid

Create a responsive webpage layout containing a top navigation bar (nav links float right) and a 4-column image gallery grid.

Requirements: Use Flexbox to align navbar items and CSS Grid to structure the gallery. Use media queries to collapse the navbar into a vertical stack and drop the gallery to 1 column on mobile devices (width < 600px).

MODULE 5

JavaScript Essentials

5.1 JS Syntax, Variables & Flow Control

JavaScript is a high-level, interpreted, programming language that conforms to the ECMAScript specification. It is the language of the web, enabling developers to build interactive, dynamic functionality directly in the user's browser.

Variables in JavaScript are declared using `let` (block-scoped, reassignable), `const` (block-scoped, read-only), or historically `var` (function-scoped).

```
// Basic variable declarations and logic
const threshold = 18;
let userAge = 20;

if (userAge ⩾ threshold) {
    console.log("Access Granted."); // Output to console
} else {
    console.log("Access Denied.");
}
```

5.2 DOM Traversing and Node Selection

The Document Object Model (DOM) is a programming interface for web documents. It represents the structure of an HTML page as a hierarchical tree of nodes, allowing languages like JavaScript to dynamically modify element styles, attributes, and content.

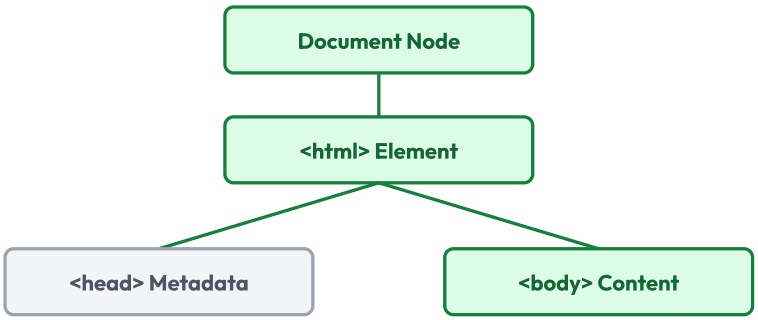

Figure 5.1: The DOM hierarchy representing HTML tags as a tree structure in memory.

Selecting Elements

JavaScript provides several methods to select HTML elements from the DOM tree:

- `document.getElementById('id')`: Selects a single element matching the specified ID.
- `document.querySelector('.class')`: Selects the first element matching a CSS selector.
- `document.querySelectorAll('.class')`: Selects all elements matching a CSS selector, returning a list of nodes.

5.3 DOM Event Listeners & Dynamic Actions

Events are actions or occurrences that happen in the browser, such as a user clicking a button, hovering over an element, or submitting a form. JavaScript can detect and respond to these events using **Event Listeners**:

```
// Select button and text element
const actionBtn = document.getElementById('my-btn');
const outputText = document.getElementById('my-text');

// Attach a click event listener
actionBtn.addEventListener('click', function() {
    outputText.textContent = "Button Clicked successfully!";
    outputText.style.color = "#15803d"; // Changes text color to green
});
```

VISUAL OUTPUT MOCKUP

Click Me Awaiting click...

5.4 Browser Storage: localStorage API

The `localStorage` object allows developers to store key-value data pairs in the browser. This data persists even when the browser is closed and reopened:

```
// Saving data to localStorage
localStorage.setItem('username', 'John Doe');

// Retrieving data from localStorage
let user = localStorage.getItem('username'); // Returns 'John Doe'
```

Module 5 - Exercises & Review Tasks

1

Debugging Task: Event Listener Reference Errors

Correct the JavaScript code segment below so that clicking the button successfully updates the heading:

```
<h1 id="title">Old Header</h1>
<button class="btn">Change Header</button>
<script>
    // Bug: Selection criteria mismatch
    let heading = document.querySelector('title');
    let btn = document.getElementById('btn');
    btn.addEventListener('click', function() {
        heading.innerHTML = "New Header";
    });
</script>
```

2

Guess the Output: Loop String Concatenation

Trace the loop iterations and state the exact text printed in the console:

```
let words = ["Web", "Design", "Rules"];
let sentence = "";
for (let i = 0; i < words.length; i++) {
    sentence += words[i] + "-";
}
console.log(sentence);
```

3

Theory Review Questions

Answer the following questions in detail:

- Explain the Document Object Model (DOM). How does the browser represent an HTML page in memory?
- Compare the variable declaration keywords `let`, `const`, and `var` in terms of scope and reassignability.
- What is an event listener? Explain how the browser captures and dispatches user actions (like click or submit).

- What is the difference between `localStorage` and session-based browser cookies in terms of capacity, persistence, and network transmissions?

Module 5 - Practical Lab Work

Lab 5.1: Dynamic To-Do List Application

Write an interactive web application that implements a simple To-Do List.

Requirements: Create an HTML page containing an input text field, an "Add Task" button, and an empty list container. Write JavaScript code that listens for clicks on the button, appends the input text as a new list item, and clears the input field.